

Software Security Framework using the Combination of Use-Misuse Diagrams and Test-Driven Development

Abdelaziz O. Akhlaif
Faculty of Information Technology
University of Benghazi, Libya
abdelaziz.akhlaif@gmail.com

Abdelsalam M. Maatuk
Faculty of Information Technology
University of Benghazi, Libya
abdelsalam.maatur@uob.edu.ly

Salwa M. Elakeili
Faculty of Information Technology
University of Benghazi, Libya
salwa.elakeili@uob.edu.ly

Abstract—In recent years, Software Security has gained an important role in the research of information systems as the increasing popularity of hacking and attacking software systems. Therefore, the need for security measures grows and many researchers have put security measures into account from the early stages of software development starting from the requirement engineering phase. Traditional security methods introduced numerous approaches to this subject from the identification of security requirements representation using different processes and techniques such as misuse case diagrams and attack trees, etc. However, these approaches focus on identifying security requirements and ignoring their inclusion in the system. This study investigates the problem of embedding security requirements within the system to ensure and improve security. A framework has been proposed to overcome this problem. The framework has been applied to a case study, i.e., Car Rental System and FileZilla FTP Server as verification and evaluation of the concept. An experimental study has been conducted to ensure that the framework can be applied in real-world applications by several software engineers with different expertise and background. A discussion about the evaluation comparison has been taken place to demonstrate the results. The results have been encouraging and prove that the proposed solution is valid.

Keywords- SRE, software engineering, requirements engineering, use cases modeling, test-driven development.

I. INTRODUCTION

Security Requirement Engineering (SRE) aims to elicit security requirements as early as possible within the requirements engineering phase along with functional requirements and constraints identifications [1]. Conventional requirements engineering (RE) approaches, generally, deal with functional requirements and constraints overlooking security requirements [2]. Such neglecting might lead to system threats such as leakage or loss of vital information, so this might result in system failure [3]. In terms of security requirements, traditional RE approaches have been used to support the representation of security threats as early as possible in software development [4, 22, 24].

Use case modeling is not enough to represent the negative requirements, which clarify the negative intent of the attackers towards the system or its users [23, 25]. Moreover, these kinds of security requirements cannot be enumerated in the late stages. Misuse case diagrams were introduced to help visualize non-functional requirements such as security, safety,

and privacy [5]. The main premise of misuse cases is to capture the hostile intent of the attacker and represent them in the simplest possible way so that the system developers could handle them earlier. Misuse cases are found efficient in this area and could provide the means to overcome threats [6][7].

After considering the security requirements, they should be implemented in a stronger and protected manner. Therefore, the Test-Driven Development (TDD) has been proposed as a development technique to make the system implementation more effective to produce a good quality product. This process as most of the XP processes is iterative, incremental, and evolutionary on every piece of a test case in each component of the software system until there are no more test cases left to satisfy. As a result, an error-free component is delivered, and so on [8]. In this context, shortcomings, such as deploying security threats to later stages remain a challenge and not fully studied.

According to the investigation described in [3], it is noticed that almost all of the approaches focus on identifying and representation of security requirements. While this might be helpful in most cases, there are still some limitations and a clear gap between the representation and the implementation of these requirements. This might lead to several problems such as misinterpretation, weak implementation, overlooked critical threats, and the most dangerous one of all is not making the line of defense from the early stages in a flexible manner.

This study aims to develop a framework called The Misuse Case and TDD Combination (MITCo). This paper introduces the framework for the implementation of security requirements using the combination of use-misuse diagrams and TDD to represent and implement software engineering security requirements in terms of early aspect. The framework consists of three stages, which are (i) the representation of security requirements with misuse case diagrams, (ii) the derivation of test cases, and (iii) the embedding of security requirements using TDD. The proposed solution has been applied and evaluated using a case study, where the results showed that the proposed framework is practical and satisfactory.

The remainder of the paper is organized as follows: Section 2 presents the related work to the software security approaches as well as misuse case modeling approaches, test-driven development approaches, and test case derivation approaches. Section 3 introduces the MITCo framework approach. Section

4 discusses some of the obtained results. Section 5 concludes the paper.

II. RELATED WORK

Different approaches are dedicated to security requirements focusing on different areas. An information flow-oriented model used for modeling privacy threats in a software system is presented in [9]. As this model is built, the framework points the analyst to where the vulnerabilities might occur so that the analyst can investigate them by listing the privacy threat types and map them to the elements defined in the system model. However, the use of the Data Flow Diagram (DFD) might cause interrelationships between entities and affect DFD elements. As a result, many communication instances of the same data might accure and need to be linked, which is logically hard to implement. Security Requirements Elicitation from Business Process (SREBP) [10] could provide the business analysts the ways to understand and represent the security requirements as Security Risk-oriented Patterns (SRPs). On the other hand, business process models may differ in the security requirements sequence in real-life scenarios. As a result, the implementation of these scenarios might be reversed or in the wrong order of events.

Misuse cases can be combined with use cases for representing security requirements [11]. Non-functional requirements, such as security requirements are hard to be represented with use cases. This is where misuse cases provide the most benefits to simplify the representation of this kind of requirement. However, the work described in [12] argued against the approach presented in [11] and introduce a set of guidelines as an anti-pattern for misuse cases. The study stated that the use of misuse cases often hides the implicit tradeoffs between security, availability, and other quality factors needed in the system.

Herrmann et al. [13] have proposed a misuse-oriented quality requirement engineering approach to identify quality requirements, whereas Joshua and Xu [14] proposed a structured approach to studying the trade-off analysis of secured systems to prove that early elicitation of security requirements using misuse cases and combining them to the architectural design may lower the cost of security risks and the tradeoffs. However, both methods identified the risks and vulnerabilities without giving an alternative solution to overcome any issues found in the system.

El-Attar et al. [15] have proposed an approach for the transformation of the misuse case descriptions gathered by the analyst and model them with mal-activity diagrams to bridge the gap at the development process in the analysis phase. However, this gap needs to be further pursued to the implementation phase to make sure that the development team makes a priority of these security requirements alongside the functional requirements.

A method for security requirements elicitation for the early integration of security within software development is presented in [16]. The method has used abuse frames to represent security threats. However, the abuse frames rarely introduce a mitigation mechanism, thus, it will be falling short of its intended goal. El-Attar [17] presented a systematic

approach by developing a set of comprehensive acceptance tests that are made for assuring security in software engineering processes. These tests are initiated and utilized at the early stages based on security requirements modeled by misuse cases. Khamaiseh et al. [18] presented an approach to derive test cases from security requirements modeled by using misuse cases and their mitigation use cases. The idea of the approach is to generate test cases to cover the normal behaviors derived from use cases and the negative behavior or attacks educed from misuse cases. However, the approach was effective considering discovering vulnerabilities that threatened security but did not provide the way to take them out.

Bystrický et al. [19] propose an approach to achieve a test-driven modularization, which is based entirely on unit testing to be combined to use case models. However, if the use case models were not interpreted correctly, some errors might accure. One of the errors is code duplication, which is going to be dreadful for the development team in the latter stages. The Misuse Case Programming (MCP) is a natural language processing technique to extract test cases from misuse case specifications [20]. The technique focuses on the negative requirements since most of the natural language techniques deal exclusively with positive requirements.

After analyzing the literature, we have found that none of the proposals provide a line of defense for the elicited security requirements.

III. THE PROPOSED FRAMEWORK

This section describes our proposed framework, which is called the MITCo. The framework embeds security requirements within the system in three main phases as shown in Fig. 1. Each phase has some systematic activities, the output of which acts as an input for the next phase. Besides, every phase is a standalone method on its own and serves a specific purpose. The idea of combining all of the phases is to achieve a bigger goal in enhancing security in software systems. Phase I tends to study the use cases and the misuse cases, which were delivered by the requirement engineers. Phase II derives a test case from the mitigation strategy, which remedies the overall look of the strategy in a more practical manner. These test cases will, later on, be bridged within the system to be applied using the TDD practice in Phase III. Before Phase I, security requirements should be addressed and specified. Besides, it is useful to link each use case to its misuse case and mitigation.

A. Phase I: Misuse Cases and their Mitigations Generation

This phase deals directly with security requirements. The misuse case represents the threat, which the system engineers create a countermeasure for. The abuser actor represents the attacker who exploits the system. Each misuse case threatens a certain use case. The threaten relationships represent the interaction between the misuse case and the use case, which about to be threatened. The mitigates relationship represents the appropriate countermeasure of each attack as shown in Fig. 2.

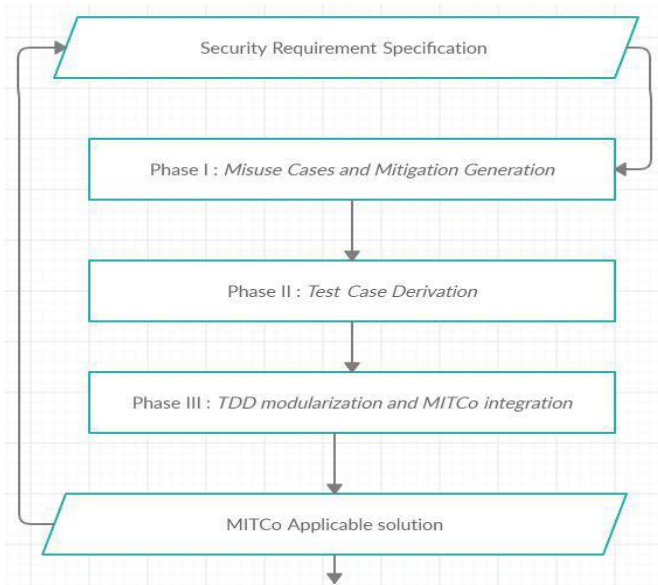


Fig. 1. The MITCo Framework

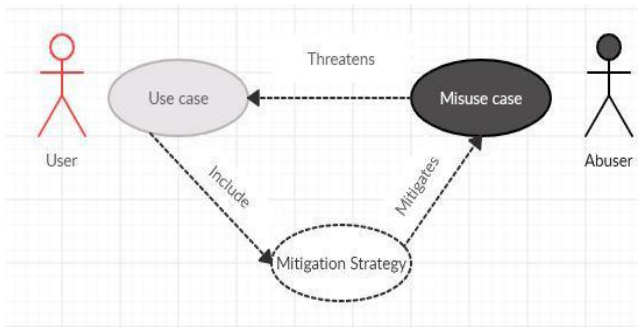


Fig. 2. Misuse Case and its Mitigation

Each misuse case should cover a certain aspect of the security requirements described in the requirements document. Misuse case diagrams are not different from use cases and could be modeled using the same tools.

Each misuse case should have a mitigation strategy to block any attack against the system. Each of these mitigation strategies should be later on dealt with as a standalone use case, which would provide a certain functionality as a security aspect. Moreover, it is useful to involve the stakeholder in the mitigation strategies to make sure that everyone agrees on them.

To show how misuse cases are derived from a use case, Table I defines a use case, and Table II illustrates a traceable means to map every use case to its misuse case. Lastly, Table III communicates every misuse case to its mitigation strategy.

TABLE I: USE CASE DEFINITION FORM

Use case	Use case identification code
Use case name	Misuse case unique name.
Actors	The user or entity who uses the system.
Goal	The purpose of the use case.
Preconditions	What should happen to trigger this event?
Post-condition	What should happen to complete this event?

TABLE II: MISUSE CASE DEFINITION FORM

Misuse case	Misuse case identification code
Misuse case	Misuse case unique name.
Actors	The abuser who attacks the system.
Goal	The purpose of the Misuse case.
Preconditions	What should happen to trigger this event?
Post-condition	What should happen after this event?
Mitigation	The counter is used to protect the user.

TABLE III: MITIGATION STRATEGY DEFINITION FORM

Mitigation use case	Mitigation use case identification code.
Mitigation case name	Mitigation case unique name.
Goal	The purpose of the Mitigation use case.
Precondition	What should happen to trigger this event?
Post-condition	What should happen to prevent this event?
Priority	How to order the importance of each mitigation strategy.

B. Phase II: Test Case Derivation

In Phase I, the mitigation strategies are described based on the misuse cases of the security requirements. In Phase II, the mitigation strategy describes how software engineers can easily formulate a test case and a test case description. A detailed description of the basic flow can be derived from the mitigation strategy along with the alternative flow. The parameters needed to trigger the test case are stated as a precondition, which might be a simple event or any source of data that starts the flow. In the case of large or complex scale software, each test case can be subdivided into a set of test cases, each of which describes a certain scenario or a viewpoint. After specifying all of the test case elements the software engineers start to implement every test case at a time. Every test case table contains the parameters shown in Table IV:

TABLE IV: DEFINING TEST CASE COMPONENTS

Test Case ID	Test Scenario	Test Steps	Expected Results	Actual Results	Pass/Fail
A unique number for each test case	The normal way the test is going to be	Test case logical flow	What should happen?	What actually happens	Whether the test case met the expected results

Test Case ID: Which represents a unique number of each test case.

Test Scenario: Describes the test case purpose with a precise name.

Test Steps: Describes the test flow with short and well-detailed steps.

Expected Results: Describes what the outcome of the normal flow of each test case should look like.

Actual Results: Describes the real-time outcome of each test case.

Pass/Fail: States whether the test case has passed or failed the test.

When all of the parameters are specified correctly, completely, and descriptively, the testing environment should be ready and effective to successfully activates the purpose of this step.

The pass or fail of the test in test cases is important for software engineers, specifically software testers. The reason for this is that the testing phase often comes late in the software development life cycle. That is why the software testers are under a lot of pressure when leaving bugs unchecked or dismissed. This might lead to a functional failure or in the worst-case scenario a system failure. However, in the MITCo framework, the test cases are not the core of testing in this case. They represent a gate to the next phase, where the first step of implementation takes place. In other words, it is safe for inexperienced engineers to make mistakes in this phase because it is very easy to trace any change or mistake to its initial state. Besides, an integration test takes place in the later stages of development. The outcomes of this phase should be clear and concise test cases, which should be the basis for the next phase. These test cases are used to formulate the needed criteria for the TDD implementation technique.

C. Phase III: TDD Modularization and MITCo Integration

This phase discusses what has been derived from the concept of TDD. Based on the investigation made regarding the usage of TDD as a schema or a concept, it is has noted that many studies have made that possible and applicable. In other words, the focus will be on the design not on coding. Besides, Phase III is the last phase in the MITCo framework, which is linking the test case derived from the misuse cases into a TDD schema to start the basic flow and iteration of each test case. This way, the design of this work will be in a close-loop starting from the user requirements toward having them within the system as shown in Fig. 3.

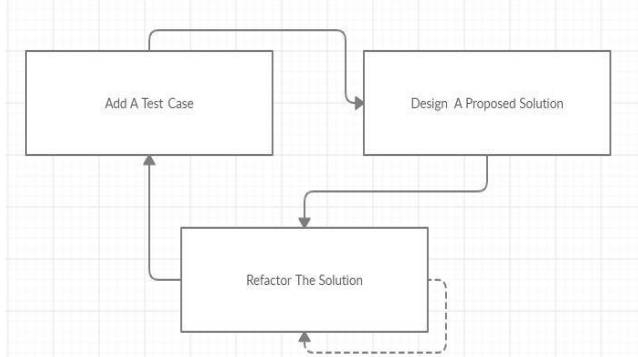


Fig. 3: Test-Driven Development Model Schema.

To achieve that several steps regarding TDD schema must be followed:

- **Add a test case:** this is the first step of each iteration made in the TDD paradigm. A test case should be described and specified carefully so, no ambiguity or misinterpretation happens.
- **Design a proposed solution:** each test case is concerned with a unique problem. Besides, to overcome this problem, a suitable solution should be designed in terms of high-level schematic statements.
- **Refactoring the solution:** this is an iterative step, where each solution described to every test case should be inspected and corrected in case of flaws or shortcomings.

This way each iteration should yield an even better solution to each test case.

- **Repeat:** the cycle is repeated with a new test case and merges the new solutions till the end of the test cases.

The MITCo framework has three main phases, each of which performs a certain purpose, which might be hard for inexperienced software engineers especially when dealing with a large or complex software system. However, the divide and conquer strategy used in this research makes it easy and effortless compared to work on each method apart.

IV. EXPERIMENTS AND RESULTS

In this study, we have conducted two experiments to validate and evaluate the proposed solution. Experiment I was for applying the proposed MITCo framework to a Car Rental System case study. The experiment has proved that the framework can be applied to real-life scenarios. Experiment II was to verify and validate the MITCo framework by its application to the FileZilla FTP server. The approach described in [18] has applied their approach to FileZilla to demonstrate how misuse cases can be used to model security aspects and how to derive test cases from these misuse cases. Hence, applying the MITCo framework on FileZilla to compare its results with the results obtained from [18] would be sufficient to evaluate the proposed framework. The advantage of applying the MITCo framework over the approach described in [18] is that the MITCo framework takes a further step as engaging the test cases derived from the mitigation strategies with TDD and be the basis of the TDD concept. This way software engineers make sure that each security requirement specified by the requirements engineers should be later on embedded within the system. Thus, the system implementation resulted from the MITCo framework, later on, is expected to be highly reliable as every kind of attack or threat is modeled and implemented simultaneously with the system functionality. Further details of our experimental studies can be found in [21].

V. CONCLUSION

In this paper, we have introduced a framework to enhance security aspects. The framework deals with the embedding of security aspects within the system from the early stages of development. The main purpose of the framework is to draw a defensive line against abusive attacks and threats. The proposed framework comprises three main phases each phase has its methods to be performed to achieve the best results. Phase I is Misuse Cases and their Mitigations Generation. Phase II is Test Case Derivation, which deals particularly with converting the misuse cases and the mitigation strategies into defined, functional test cases. Phase III is concerned with driving the development process with TDD based on the test cases derived from Phase II. This integration is expected to perform as a shield of countermeasures against hacks and misuse. The framework has been evaluated using two case studies. The results demonstrated that the MITCo framework is feasible and acceptable. To conclude, the investigation into security requirements engineering has shown that security

aspects in requirements analysis are not fully pursued in the development process as the regular requirements. As a result, developers neglect security aspects and build software systems without putting the countermeasures needed to overcome threats and attacks. In the available literature, we showed that the existing approaches deal with the identification and representation of security requirements. Thus, a framework desired to cope with this issue is required.

REFERENCES

- [1] M. Donatas, and R. Butleris, "Integrating security requirements engineering into MBSE: Profile and guidelines" *Security and Communication Networks* (2020), 2020.
- [2] G. McGraw, *Software Security: Building Security In*, Upper Saddle River, NJ: Addison-Wesley, 2006.
- [3] M. Magne, "Industrial experiences with misuse cases," Master's thesis, Institutt for datateknikk og informasjonsvitenskap, The Dep. of Computer Technology and Informatics, Norwegian University of Science and Technology, 2006.
- [4] A. Abasi-Amefon, M. Raimundas, and N. Alexander. "Security Risk Management in E-commerce Systems: A Threat-driven Approach." *Baltic Journal of Modern Computing*, 2020.
- [5] S. Guttorm, and O. Andreas, "Eliciting security requirements with misuse cases": *Requirements engineering*, 2005.
- [6] O. Takao, and T. Hidehiko, "Identifying security aspects in early development stages". *Third International Conference on Availability, Reliability and Security*. IEEE, 2008.
- [7] M. Othmar, and E. Isao, "Security Oriented Malicious Activity Diagrams to Support Information Systems Security", *31st International Conference on Advanced Information Networking and Applications Workshops (WAINA)*, pp. 74-81, 2017.
- [8] B. Kent, "Test-driven development: by example" Upper Saddle River, NJ: Addison-Wesley, 2003.
- [9] D. Mina, W. Kim, S. Riccardo, P. Bart, J. Wouter "A privacy threat analysis framework: supporting the elicitation and fulfillment of privacy requirements". *Requirements Engineering*, vol. 16, no. 1, pp. 3—32, 2011.
- [10] M. Raimundas, N. Alex, and S. Silver, "Security requirements elicitation from airline turnaround processes," *Business & Information Systems Engineering*, vol. 60, no. 1, pp. 3—20, 2018.
- [11] S. Guttorm, and O. Andreas, "Eliciting security requirements with misuse cases," *Requirements Engineering*, vol. 10, no. 1, pp. 34—44, 2005.
- [12] Dashti, M. Torabi, and R. Saša, "An Anti-pattern for Misuse Cases.", *Computer Security*, Springer, pp. 250—261, 2017.
- [13] H. Andrea, P. Barbara, "MOQARE: misuse-oriented quality requirements engineering", *Requirements Engineering*, vol. 13, no. 1, pp. 73—86, 2008.
- [14] P. Joshua, and X. Dianxiang, "Trade-off Analysis of Misuse Case-based Secure Software Architectures: A Case Study", *Proceedings of the 3rd International Workshop on Modelling, Simulation, Verification and Validation of Enterprise Information Systems*, 2005.
- [15] E. Mohamed, "From misuse cases to mal-activity diagrams: bridging the gap between functional security analysis and design", *Software & Systems Modeling*, vol. 13, no. 1, pp. 173—190, 2014.
- [16] E. Hassan, and E. Sherif. "Capturing security requirements for software systems.", *Journal of advanced research*, vol. 5, no. 4, pp. 463—472, 2014.
- [17] E. Mohamed and A. Hezam, "Using security robustness analysis for early-stage validation of functional security requirements", *Requirements Engineering*, vol. 21, no. 1, pp. 1—27, 2016.
- [18] K. Samer, and X. Dianxiang, "Software security testing via misuse case modeling", *2017 IEEE 15th Intl Conf on Dependable, Autonomic and Secure Computing, 15th Intl Conf on Pervasive Intelligence and Computing, 3rd Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress*, IEEE, pp. 534—541, 2017.
- [19] B. Michal, and V. Valentino, "Use case-driven modularization as a basis for test-driven modularization". *2017 Federated Conference on Computer Science and Information Systems (FedCSIS)*, IEEE, pp. 693—596, 2017.
- [20] P. Mai, F. Pastore, and A. Göknil, L. Briand, "MCP: A Security Testing Tool Driven by Requirements", *the 41st International Conference on Software Engineering: Companion Proceedings*, 2019, Montreal, QC, Canada, 2019, ICSE, pp. 55--58, 2019.
- [21] A. Akhlaif, O. "Improving Software Security using the Combination of Use-Misuse Diagrams and Test-Driven Development". *University of Benghazi, Libya*, 2021.
- [22] E. A. Abdelnabi, A. M. Maatuk, T. M. Abdelaziz and S. M. Elakeili, "Generating UML Class Diagram using NLP Techniques and Heuristic Rules," *2020, 20th International Conference on Sciences and Techniques of Automatic Control and Computer Engineering (STA)*, Monastir, Tunisia, 2020, pp. 277-282, doi: 10.1109/STA50679.2020.9329301.
- [23] S. F. Alshareef, A. M. Maatuk, T. M. Abdelaziz, and M. Hagal. 2020. "Validation Framework for Aspectual Requirements Engineering (ValFAR)." *6th International Conference on Engineering & MIS 2020 (ICEMIS'20)*. Association for Computing Machinery, New York, NY, USA, Article 42, 1–7. DOI:<https://doi.org/10.1145/3410352.3410777>.
- [24] A. Elghadhafi, T. M. Abdelaziz and A. M. Maatuk. 2018. "A Novel Approach for Improving the Quality of Software Code using Reverse Engineering." *ICEMIS '18*, Turkey, ACM. DOI: <https://doi.org/10.1145/3234698.3234729>
- [25] T. M. Abdelaziz, A. M. Maatuk and F. Rajab. "An Approach to Improvement the Usability in Software Products". *International Journal of Software Engineering & Applications (IJSEA)*, Vol.7, No.2, March 2016. DOI : 10.5121/ijsea.2016.7202 11
- [26] A. O. Mohammed, Z. A. Abdelnabi, A. M. Maatuk and A. S. Abdalla. "An Experimental Study on Detecting Semantic Defects in Object-Oriented Programs using Software Reading Techniques", *ACM International Conference on Engineering & MIS (ICEMIS '15)*, ACM, New York, NY, USA, Article 24, 6 pages. DOI=<http://dx.doi.org/10.1145/2832987.2833025>